Tutorial: Programação Orientada a Objetos (POO) em C++ — Guia passo a passo

Introdução

A **Programação Orientada a Objetos (POO)** é um paradigma que organiza o código em *objetos* — estruturas que combinam **dados** (atributos) e **comportamentos** (métodos). Em C++, POO é central para escrever código modular, reutilizável e fácil de manter.

Quatro pilares da POO

- **Encapsulamento** — esconder detalhes internos e oferecer uma interface clara.
- **Abstração** — representar entidades do domínio com classes que modelam o comportamento essencial.
- **Herança** — criar novas classes a partir de outras, reutilizando código.
- **Polimorfismo** — permitir que objetos de tipos diferentes sejam usados através de uma interface comum.

---

1. Classes e objetos

Classe é o molde;

objeto é a instância.

```
#include <iostream>
#include <string>

class Pessoa {
public:
    std::string nome;
    int idade;

    void apresentar() const {
        std::cout << "Olá, eu sou " << nome << ", " << idade << " anos\n";
    }
};

int main() {
    Pessoa p;       // objeto criado na stack
    p.nome = "Luiz";
    p.idade = 25;
    p.apresentar();
    return 0;
}
```

Pontos importantes

- `public` / `private` / `protected` controlam visibilidade
- public → pode ser acessado de qualquer lugar (dentro ou fora da classe).
- private → só pode ser acessado dentro da própria classe (nem classes derivadas podem acessar).
- protected → pode ser acessado na própria classe e nas classes derivadas, mas não diretamente fora delas.

```
#include <iostream>
using namespace std;

class Exemplo {
public:
    int publico; // Qualquer código pode acessar

    void mostrar() {
        cout << "Publico: " << publico << endl;
        cout << "Privado: " << privado << endl;
        cout << "Protegido: " << protegido << endl;
    }

private:
    int privado; // Só a classe Exemplo pode acessar

protected:
    int protegido; // Pode ser acessado pela classe e pelas derivadas

public:
    // Construtor para inicializar
    Exemplo(int p, int pr, int pt) {
        publico = p;
        privado = pr;
        protegido = pt;
    }
};

// Classe derivada
class Derivada : public Exemplo {
public:
    Derivada(int p, int pr, int pt) : Exemplo(p, pr, pt) {}

    void acessarProtegido() {
        // protegido pode ser acessado aqui porque é protected
        cout << "Protegido acessado na classe derivada: " << protegido << endl;

        // privado NÃO pode ser acessado aqui
        // cout << privado; ❌ (erro de compilação)
    }
};

int main() {
    Exemplo obj(10, 20, 30);
```

```
    // Public pode ser acessado diretamente
    cout << "Acessando publico direto: " << obj.publico << endl;

    // Private NÃO pode ser acessado direto
    // cout << obj.privado; ❌ (erro de compilação)

    // Protected também NÃO pode ser acessado direto
    // cout << obj.protegido; ❌ (erro de compilação)

    // Mas podemos acessar todos dentro de um método público da classe
    obj.mostrar();

    // Classe derivada pode acessar o protected
    Derivada d(1, 2, 3);
    d.acessarProtegido();

    return 0;
}
```

---

2. Construtores e destrutores

- **Construtores** inicializam objetos;
- **Destrutores** liberam recursos ao final da vida do objeto.

```
#include <iostream>
#include <string>

class Pessoa {
public:
    std::string nome;
    int idade;

    Pessoa(const std::string& n, int i) : nome(n), idade(i) {
        std::cout << "Construtor: " << nome << "\n";
    }
    ~Pessoa() {
        std::cout << "Destrutor: " << nome << "\n";
    }

    void apresentar() const {
        std::cout << nome << " — " << idade << " anos\n";
    }
};

int main() {
    Pessoa a("Ana", 30);
    a.apresentar();
}
```

Dicas

- Use **member initializer lists** (`: nome(n), idade(i)`) para inicializar membros.
- O destrutor `~Classe()` é chamado automaticamente quando o objeto sai de escopo.

---

3. Encapsulamento: getters e setters

Eles fazem parte do **encapsulamento**: Coloque atributos sensíveis em `private` e controle acesso com métodos públicos.

getter → retorna o valor de um atributo privado/protegido.

setter → altera o valor de um atributo privado/protegido (geralmente com validação).

```
#include <iostream>
using namespace std;

class Pessoa {
private:
    string nome;  // atributo privado
    int idade;

public:
    // Setter do nome
    void setNome(const string& novoNome) {
        if (novoNome.empty()) {
            cout << "Nome invalido!" << endl;
        } else {
            nome = novoNome;
        }
    }

    // Getter do nome
    string getNome() const {
        return nome;
    }

    // Setter da idade
    void setIdade(int novaIdade) {
        if (novaIdade < 0) {
            cout << "Idade invalida!" << endl;
        } else {
            idade = novaIdade;
        }
    }

    // Getter da idade
    int getIdade() const {
        return idade;
```

```
    }
};

int main() {
    Pessoa p;

    p.setNome("Luiz");
    p.setIdade(20);

    cout << "Nome: " << p.getNome() << endl;
    cout << "Idade: " << p.getIdade() << endl;

    p.setIdade(-5); // inválido
}
```

Motivo: validação centralizada e proteção do estado interno.

---

4. Métodos `const`, `static` e `inline`

- `void foo() const` — garante que o método não altera o estado do objeto.
- `static` — membro ligado à classe, não à instância.
- `inline` — dica ao compilador para expandir (normalmente em headers).

Métodos const Dizem ao compilador que o método não pode modificar os atributos da instância. Só podem acessar métodos/variáveis que também sejam const.

```
#include <iostream>
using namespace std;

class ConstExemplo {
private:
    int valor;
public:
    ConstExemplo(int v) : valor(v) {}

    int getValor() const {  // Método const
        // valor++; ❌ (ERRO: não pode modificar dentro de const)
        return valor;
    }

    void setValor(int v) {  // Método normal
        valor = v;
    }
};

int main() {
    const ConstExemplo obj(10);
    cout << obj.getValor() << endl;  // ☑ permitido
```

```
    // obj.setValor(20); ❌ (ERRO: objeto const só pode chamar métodos const)
}
```

Métodos static Pertencem à classe e não ao objeto.

Não precisam de instância para serem chamados.

Não podem acessar atributos/métodos **não-estáticos** (porque não têm um objeto associado).

```
#include <iostream>
using namespace std;

class StaticExemplo {
private:
    static int contador; // variável estática compartilhada por todos
public:
    StaticExemplo() { contador++; }

    static int getContador() { // método static
        return contador;
    }
};

// Definição da variável estática
int StaticExemplo::contador = 0;

int main() {
    StaticExemplo a, b, c;
    cout << "Objetos criados: " << StaticExemplo::getContador() << endl;
}
```

Métodos inline Pedem ao compilador para substituir a chamada do método pelo código dele (evita custo de chamada de função).

Útil em funções muito pequenas.

O compilador pode ignorar o pedido se achar melhor.

```
#include <iostream>
using namespace std;

class InlineExemplo {
public:
    inline int quadrado(int x) { // função inline
        return x * x;
    }
};

int main() {
```

```
    InlineExemplo obj;
    cout << "Quadrado de 5: " << obj.quadrado(5) << endl;
}
```

5. Herança

Cria uma relação `é-um` entre classes.

```
#include <iostream>

class Animal {
public:
    virtual void falar() const { std::cout << "Som de animal\n"; }
};

class Cachorro : public Animal {
public:
    void falar() const override { std::cout << "Au au!\n"; }
};

int main() {
    Cachorro c;
    c.falar(); // "Au au!"
}
```

Notas

- `public` herança é a mais comum: `class Derived : public Base`.

---

6. Polimorfismo (estático vs dinâmico)

- **Estático (compile-time)**: overloading, templates.

Também chamado de sobrecarga (overloading) e templates.

A escolha de qual função usar é feita no momento da compilação

```
#include <iostream>
using namespace std;

class Calculadora {
public:
    // Soma de inteiros
    int soma(int a, int b) {
        return a + b;
    }
```

```
    // Sobrecarga: soma de doubles
    double soma(double a, double b) {
        return a + b;
    }

    // Sobrecarga: soma de três valores
    int soma(int a, int b, int c) {
        return a + b + c;
    }
};

int main() {
    Calculadora calc;
    cout << calc.soma(2, 3) << endl;        // chama soma(int, int)
    cout << calc.soma(2.5, 3.5) << endl;    // chama soma(double, double)
    cout << calc.soma(1, 2, 3) << endl;     // chama soma(int, int, int)
}
```

- **Dinâmico (run-time)**: `virtual` + chamadas via referência ou ponteiro.

Exemplo usando referência (sem precisar de ponteiro):

Também chamado de sobrescrita (overriding). A escolha de qual método chamar acontece durante a execução, baseado no tipo real do objeto (e não no tipo do ponteiro/referência).

Isso só funciona com herança + métodos virtuais.

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void falar() {   // método virtual
        cout << "Animal faz um som" << endl;
    }
};

class Cachorro : public Animal {
public:
    void falar() override {  // sobrescreve
        cout << "Cachorro late: Au Au!" << endl;
    }
};

class Gato : public Animal {
public:
    void falar() override {
        cout << "Gato mia: Miau!" << endl;
    }
};
```

```
int main() {
    Animal* a1 = new Cachorro();
    Animal* a2 = new Gato();

    a1->falar();  // escolhe em tempo de execução: Cachorro
    a2->falar();  // escolhe em tempo de execução: Gato

    delete a1;
    delete a2;
}
```

---

7. Classes abstratas e interfaces

Classes Abstratas São classes que não podem ser instanciadas diretamente.

Servem como modelo/base para outras classes.

Possuem pelo menos um método virtual puro (=0).

```
virtual void metodo() = 0;
```

Exemplo: Classe Abstrata

```
#include <iostream>
using namespace std;

// Classe abstrata
class Animal {
public:
    virtual void falar() = 0;   // método virtual puro (obrigatório sobrescrever)
    virtual void mover() = 0;   // pode ter vários métodos abstratos

    // Pode ter métodos normais também
    void dormir() {
        cout << "Zzz..." << endl;
    }
};

// Classe derivada deve implementar os métodos abstratos
class Cachorro : public Animal {
public:
    void falar() override {
        cout << "Au Au!" << endl;
    }
    void mover() override {
```

```
        cout << "Cachorro correndo..." << endl;
    }
};

class Gato : public Animal {
public:
    void falar() override {
        cout << "Miau!" << endl;
    }
    void mover() override {
        cout << "Gato andando silenciosamente..." << endl;
    }
};

int main() {
    // Animal a; ❌ (não pode instanciar classe abstrata)
    Animal* a1 = new Cachorro();
    Animal* a2 = new Gato();

    a1->falar();  // Au Au!
    a2->falar();  // Miau!
    a1->dormir(); // método concreto herdado
    a2->dormir();

    delete a1;
    delete a2;
}
```

Interfaces em C++

Diferente de Java/C#, C++ não tem a palavra-chave interface. Mas podemos simular interfaces criando uma classe somente com métodos virtuais puros.

```
#include <iostream>
using namespace std;

// Interface
class IForma {
public:
    virtual double area() = 0;
    virtual double perimetro() = 0;
    virtual ~IForma() {} // destrutor virtual é boa prática
};

// Implementação concreta
class Circulo : public IForma {
private:
    double raio;
public:
    Circulo(double r) : raio(r) {}
```

```
    double area() override {
        return 3.14 * raio * raio;
    }
    double perimetro() override {
        return 2 * 3.14 * raio;
    }
};

class Quadrado : public IForma {
private:
    double lado;
public:
    Quadrado(double l) : lado(l) {}

    double area() override {
        return lado * lado;
    }
    double perimetro() override {
        return 4 * lado;
    }
};

int main() {
    IForma* f1 = new Circulo(5);
    IForma* f2 = new Quadrado(4);

    cout << "Area do circulo: " << f1->area() << endl;
    cout << "Perimetro do quadrado: " << f2->perimetro() << endl;

    delete f1;
    delete f2;
}
```

---

8. Composição vs Herança

- **Herança**: Na herança, uma classe (derivada) herda atributos e métodos de outra classe (base). Isso cria uma relação "é um" (IS-A).

```
#include <iostream>
using namespace std;

// Classe base
class Animal {
public:
    void comer() {
        cout << "O animal está comendo." << endl;
    }
```

```
};

// Classe derivada (herda de Animal)
class Cachorro : public Animal {
public:
    void latir() {
        cout << "O cachorro está latindo." << endl;
    }
};

int main() {
    Cachorro c;
    c.comer();  // Método herdado de Animal
    c.latir();  // Método próprio de Cachorro
    return 0;
}
```

- **Composição**: Na composição, uma classe contém objetos de outras classes como atributos. Isso cria uma relação "tem um" (HAS-A).

Exemplo de composição (Turma tem Alunos):

```
#include <iostream>
using namespace std;

// Classe Motor
class Motor {
public:
    void ligar() {
        cout << "O motor está ligado." << endl;
    }
};

// Classe Carro tem um Motor (composição)
class Carro {
private:
    Motor motor; // Objeto como atributo
public:
    void ligarCarro() {
        cout << "Ligando o carro..." << endl;
        motor.ligar(); // Usando o objeto Motor
    }
};

int main() {
    Carro carro;
    carro.ligarCarro(); // Usa Motor indiretamente
    return 0;
}
```

Diferença chave

- Herança: usada quando a classe derivada é um tipo especializado da base (Cachorro é um Animal).
- Composição: usada quando a classe precisa de componentes para funcionar (Carro tem um Motor).

---

9. Regra dos 3 / Regra dos 5 (visão geral)

Se sua classe gerencia recursos (memória dinâmica, handles), você precisa controlar cópia e destruição:

- **Regra dos 3**: se implementar/destrutor, copie-construtor ou operador=, provavelmente precisará dos três.
- **Regra dos 5**: com C++11, considere também move-construtor e move-assign.

> **Observação:** essa seção é *avançada* e exige conhecimento sobre alocação dinâmica (ponteiros). Se preferir, estudamos isso após dominar POO.

---

10. Boas práticas em POO C++

- Prefira inicializar membros com *member initializer lists*.
- Mantenha membros `private` por padrão.
- Favor composição sobre herança quando possível.
- Use `const` nos métodos que não modificam o objeto.
- Separe declarações (`.h`) e implementações (`.cpp`).
- Documente a responsabilidade de cada classe (SRP — Single Responsibility Principle).

---

11. Exercícios (etapa a etapa)

Os exercícios abaixo são progressivos: comece pelo 1 e avance.

Exercício 1 — Retângulo (nível iniciante)

Objetivo: criar uma classe `Retangulo` com atributos privados `largura` e `altura`.

Etapas:

1. Declare a classe com atributos privados e um construtor público que inicializa largura e altura.
2. Implemente métodos `double area() const` e `double perimetro() const`.
3. No `main()`, crie um `Retangulo r(3.0, 4.0);` e imprima área e perímetro.
4. Adicione validação no construtor (largura/altura devem ser > 0). Se inválido, defina como 0.
5. Bônus: sobrecarregue `operator<<` para imprimir o retângulo.

Critério de aceitação: `Retangulo r(3,4)` deve imprimir `Área: 12` e `Perímetro: 14`.

Exercício 2 — ContaBancaria (iniciante → intermediário)

Objetivo: encapsular um saldo e operações básicas.

Etapas:

1. Crie `class ContaBancaria` com `private: double saldo;` e um construtor.
2. Implemente `depositar(double)`, `bool sacar(double)` (retorna `true` se OK) e `double getSaldo() const`.
3. No `main()`, simule depósitos e saques e verifique o saldo final.
4. Bônus: implemente método `transferir(ContaBancaria& destino, double valor)`.

Critério de aceitação: transferir 50 de uma conta com saldo 100 para outra deve resultar em 50 e 150.

Exercício 3 — Polimorfismo com Formas (intermediário)

Objetivo: criar hierarquia de `Forma` e usar polimorfismo via referências.

Etapas:

1. Implemente `class Forma` com `virtual double area() const = 0;`.
2. Crie `Retangulo` e `Circulo` que implementem `area()`.
3. Implemente função `void imprimeArea(const Forma& f)` que mostra a área.
4. No `main()`, crie `Retangulo r(3,4); Circulo c(2);` e chame `imprimeArea(r); imprimeArea(c);`.

Critério de aceitação: as áreas devem corresponder às fórmulas conhecidas.

Exercício 4 — Funcionários (abstração & herança)

Objetivo: modelar funcionários com diferentes tipos de pagamento.

Etapas:

1. Crie `class Funcionario` abstrata com `virtual double calcularSalario() const = 0;`.
2. Implemente `FuncionarioCLT` (salário fixo + benefícios) e `FuncionarioPJ` (horas * valorHora).
3. Faça um `vector` de objetos (por valor ou por referência) e imprima salário total da empresa.

Observação: para coleções heterogêneas, você pode armazenar ponteiros ou referências. Se quiser evitar ponteiros, calcule salários diretamente em variáveis locais.

Exercício 5 — Composição (intermediário)

Objetivo: criar `Turma` que contém `Aluno`s por composição.

Etapas:

1. Implemente `Aluno` com `nome` e `nota`.
2. Implemente `Turma` com `std::vector<Aluno>` e métodos para adicionar aluno e calcular média.

3. No `main()`, preencha a turma e calcule a média.

Exercício 6 — *Opcional avançado*: Regra dos 3 (requer ponteiros)

Objetivo: entender por que a cópia automática pode quebrar quando há recursos alocados.

Etapas:

1. Crie `class Buffer` que aloca um `int* dados` no construtor e libera no destrutor.
2. Compile e observe comportamento ao copiar objetos (cópia rasa — *shallow copy*).
3. Implemente copy-constructor e operador de atribuição para fazer *deep copy*.